



Internship Presentation

KALI NANGIA

REPORTING MANAGER

Karan Sir

TEAM MEMBERS

Abhishek, Akshay

Index of Activities

Press  to navigate sequentially through Summaries, Data Flows, Use Cases, and Sequence Diagrams for all 7 activities

01. Setup, Inception & Basic Automation

Gemini 3.5 Flash speaker-tagged transcripts (Slides 3-6)

02. Baseline Benchmarking & Local LLM

Local Gemma quantized configurations and memory audits (Slides 7-10)

03. Fine-Tuning Setup & Training

Whisper Medium vs. Whisper Turbo QLoRA training (Slides 11-14)

04. Speaker Diarization & Audio Splitting

Pyannote speaker tracking, timeline refinement, & turn splitting (Slides 15-18)

05. Pipeline Integration

Unified monolithic notebook pipeline execution flow (Slides 19-22)

06. Modularization, Migration & API Fallbacks

Deconstructing monolith into 6 atomic notebooks (Slides 23-26)

07. Local LLM Scaling & Benchmarking

Gemma 2B/4B VRAM profiling for offline fallbacks (Slides 27-30)

01. Setup, Inception & Basic Automation — Summary

June 8 – June 9 (Day 1 – Day 2) | 2 Days Active

Divided 30 Krishi Darshan videos among three team members. Realizing manual transcription was too tedious, I built a notebook to pivot transcription tasks to Gemini's API.

 SOURCE CODE REPOSITORY

Check pipeline code here: [Extraction Pipeline](#)

Gemini 3.5 Flash API

Bulk Processing

WHISPER V3 TURBO (BASELINE)

- **Intro Misspellings:** Missed dialects: तिथी दर्शन कार्यक्रम, स्वाव लंबी.
- **Looping Glitch:** Acoustic loops (e.g., "कर दो कर दो कर...").

GEMINI 3.5 FLASH (API)

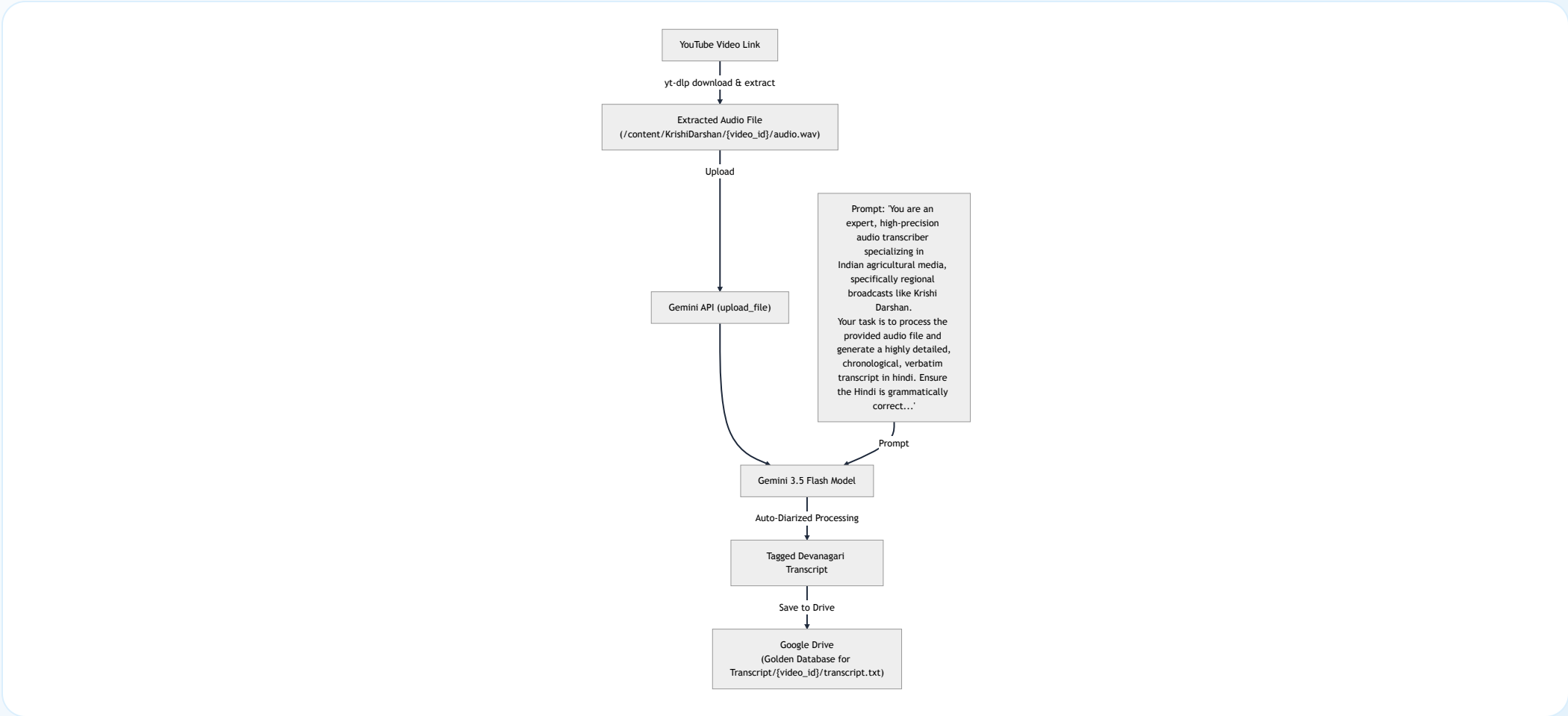
- **Clean Standard Hindi:** Perfect spelling: कृषि दर्शन, स्वावलंबी.
- **No Loop Glitch:** Clean transitions across content segments.

```
# 1. Download and extract audio from YouTube
Download_Audio(video_id) # Saves to /content/KrishiDarshan/{video_id}/audio.wav

# 2. Upload audio and generate verbatim transcript
audio_file = genai.upload_file(f"/content/KrishiDarshan/{video_id}/audio.wav")
model = genai.GenerativeModel("gemini-3.5-flash")
response = model.generate_content([
    audio_file,
    "You are an expert, high-precision audio transcriber specializing in Indian
    agricultural media, specifically regional broadcasts like Krishi Darshan. Your
```

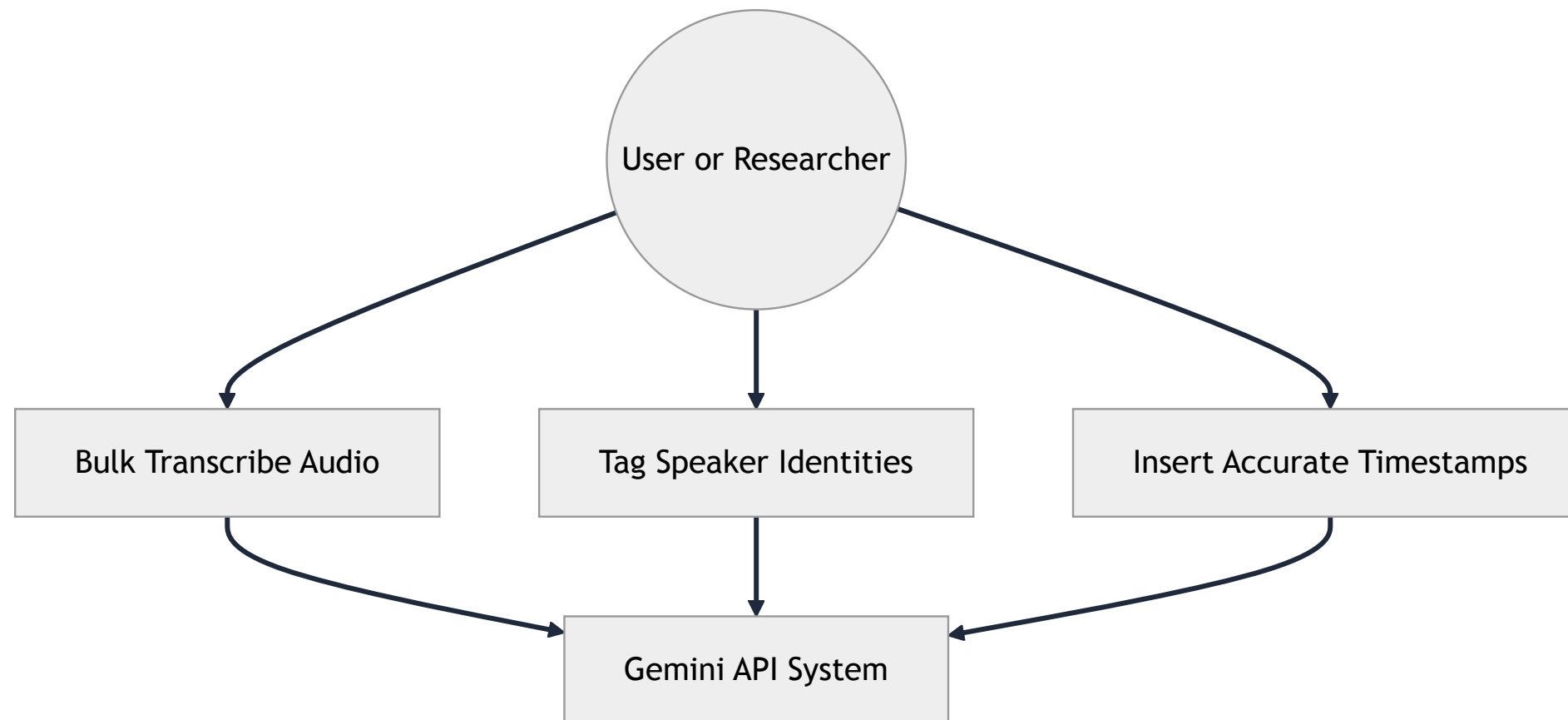
01. Setup, Inception & Basic Automation — Data Flow

June 8 – June 9 (Day 1 – Day 2) | Data Pipeline mapping inputs, API calls and output formats



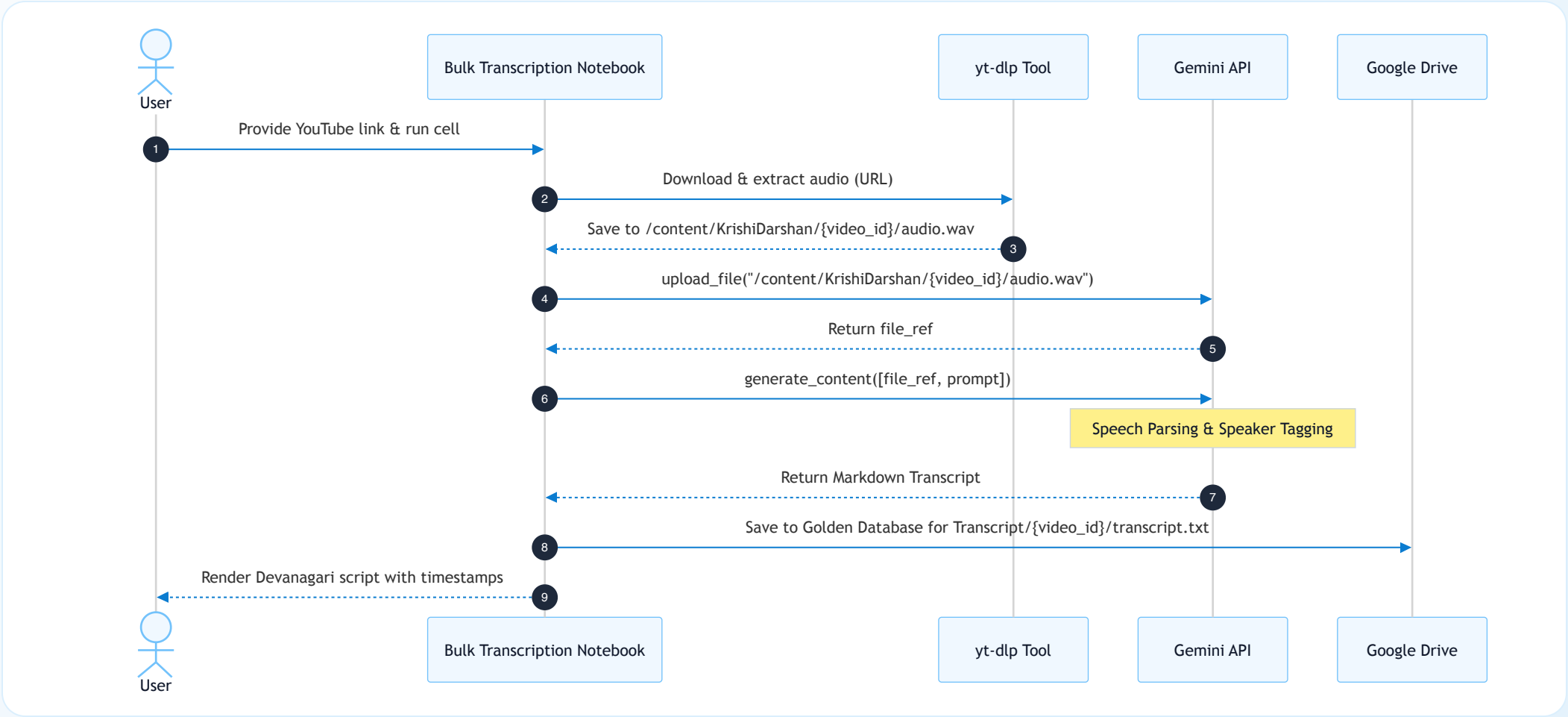
01. Setup, Inception & Basic Automation — Use Case

June 8 – June 9 (Day 1 – Day 2) | Main goals, actions, and entities involved



01. Setup, Inception & Basic Automation — Sequence Diagram

June 8 – June 9 (Day 1 – Day 2) | Step-by-step function calls & responses



02. Baseline Benchmarking & Local LLM — Summary

June 10 – June 11 (Day 3 – Day 4) | 2 Days Active

We evaluated local multimodal transcription and text correction capabilities using quantized Gemma 12B model variants to determine boundary constraints.

Memory Resource Audit

- ****System RAM Crash:**** Raw 12B model (24 GB files) overloaded Colab's 12.7 GB system memory during download.
- ****VRAM VMemory Crash:**** Quantized 4-bit model files fit in VRAM, but generating tokens on 185s clips caused calculations that flooded graphics memory.

Gemma 12B

NF4 Quantization



[Source Code Repository](#)

Check ASR notebook code here: [Gemma ASR Notebook](#)

```
from transformers import BitsAndBytesConfig

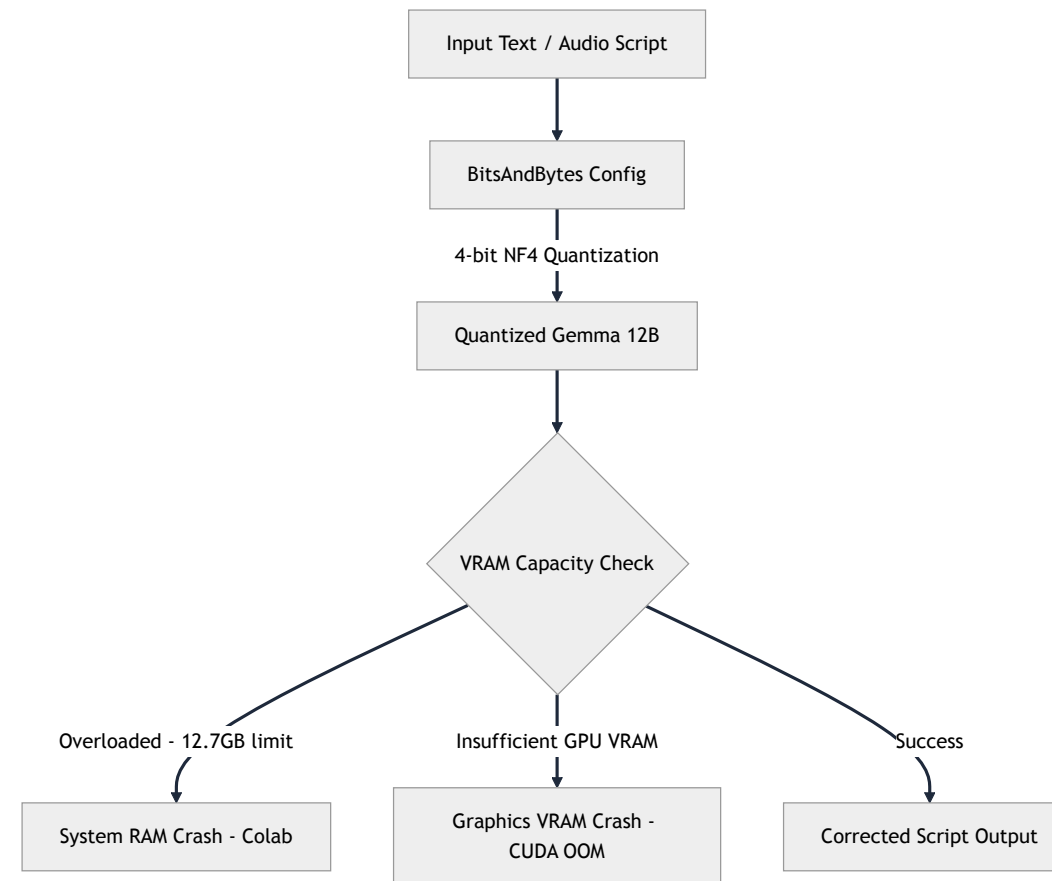
# 4-bit configuration to fit model into T4 GPU VRAM
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)
model = AutoModelForCausalLM.from_pretrained(
    ...,
    quantization_config=bnb_config,
    low_cpu_mem_usage=True
)
```

Model Size Parameters:

- Raw Model: ~24.0 GB (Requires high-memory VMs)
- Quantized (4-bit NF4): ~7.5 GB (Fits within Colab T4 16GB GPU VRAM)

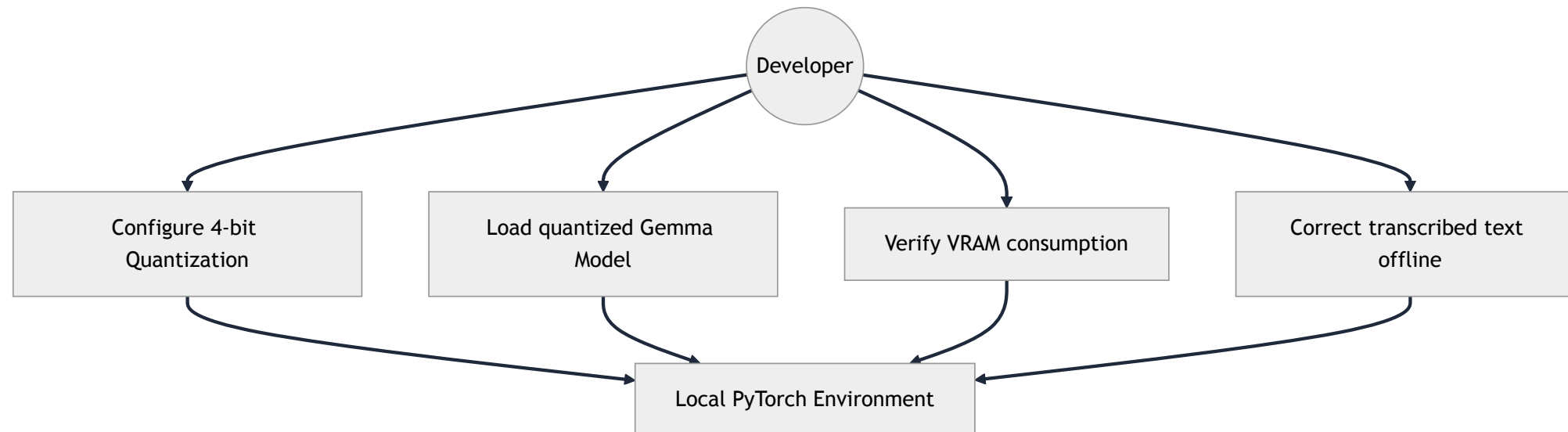
02. Baseline Benchmarking & Local LLM — Data Flow

June 10 – June 11 (Day 3 – Day 4) | Data flow under memory constraints



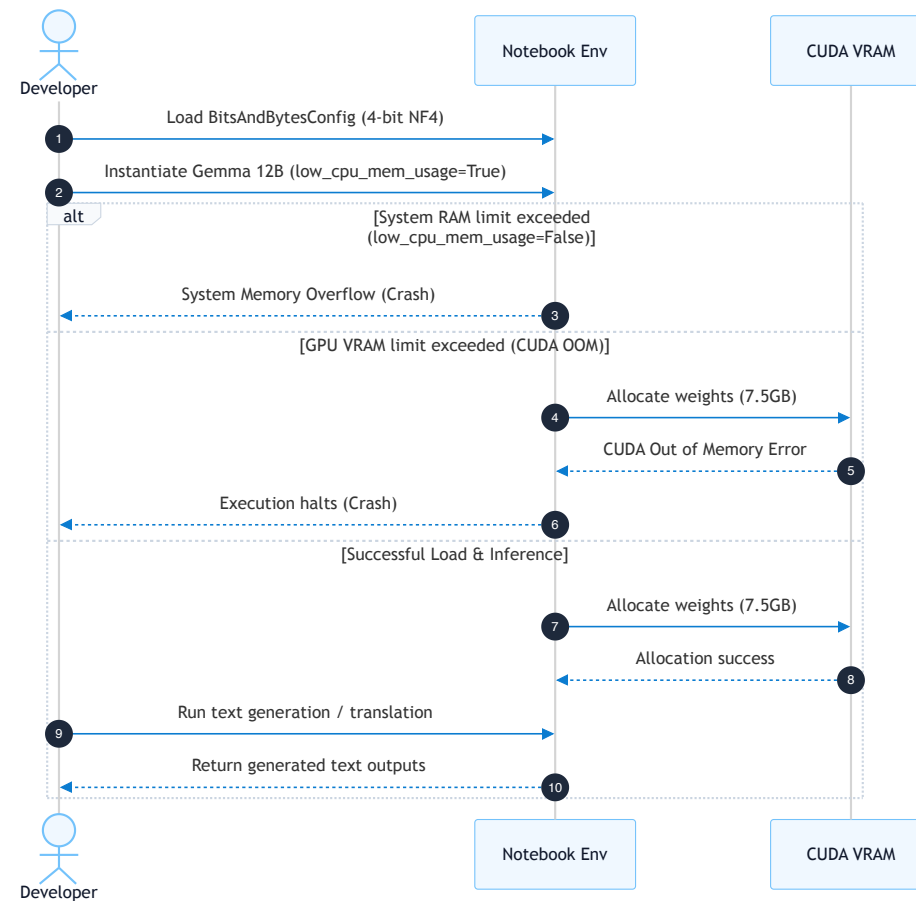
02. Baseline Benchmarking & Local LLM — Use Case

June 10 – June 11 (Day 3 – Day 4) | Developer objectives and local pipeline constraints



02. Baseline Benchmarking & Local LLM — Sequence Diagram

June 10 – June 11 (Day 3 – Day 4) | CUDA and System memory check sequence



03. Fine-Tuning Setup & Training — Summary

June 12 – June 17 (Day 5 – Day 9) | 6 Days Active

We analyzed Whisper Large V3 baseline failures, launched a QLoRA fine-tuning training environment, and completed Whisper model training on Gurmukhi script datasets.

WHISPER MEDIUM

- **WER:** 0.8272 (Baseline)
- **Score:** 5/10 (Qualitative)
- **Data:** 8 hrs Gurmukhi speech
- **Val Loss:** Stable 0.36 → 0.44 (1k steps)
- **Time:** 6.4 hrs training (~8.0 hrs on T4 GPU)

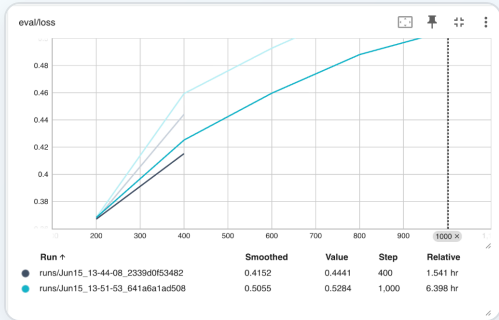
WHISPER TURBO

- **WER:** 0.8020 (Optimal)
- **Score:** 7/10 (Qualitative)
- **Data:** Scaled 8 → 40 hrs speech
- **Val Loss:** Spiked on scaling, then dec (3.2k steps)
- **Time:** 10.1 hrs training (~12.0 hrs on T4 GPU)

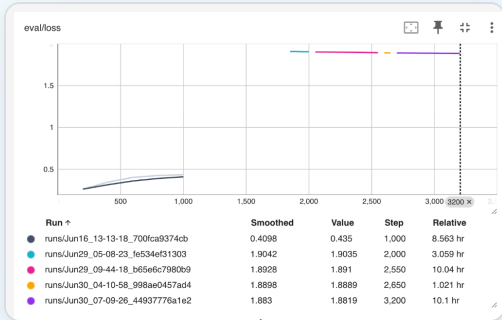
Whisper Large V3 Turbo

QLoRA PEFT

Whisper Medium (8 hrs Data)



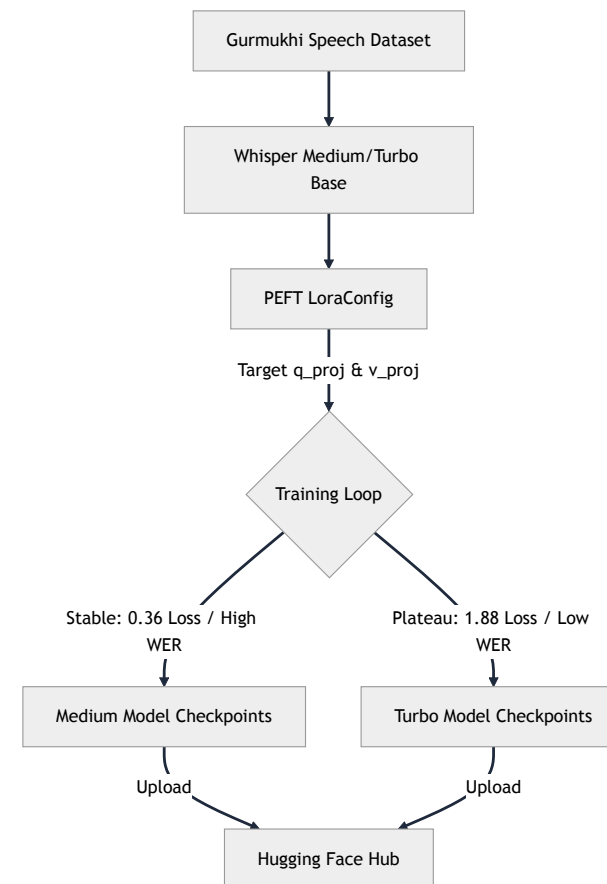
Whisper Turbo (8 → 40 hrs Scaling)



```
lora_config = LoraConfig(  
    lora_alpha=64, target_modules=["q_proj", "v_proj"], lora_dropout=0.05  
)  
model = get_peft_model(model, lora_config)
```

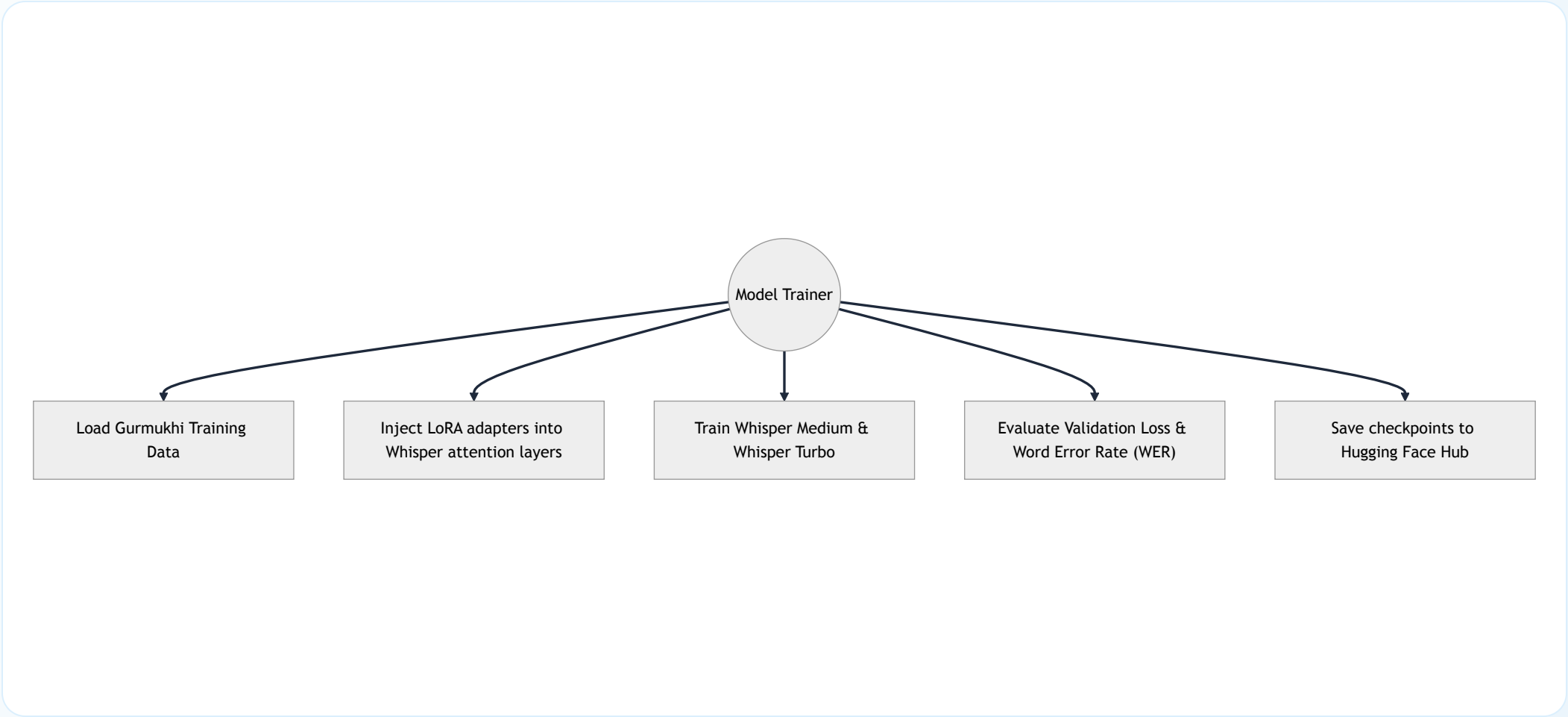
03. Fine-Tuning Setup & Training — Data Flow

June 12 – June 17 (Day 5 – Day 9) | Training dataset flow with PEFT insertion



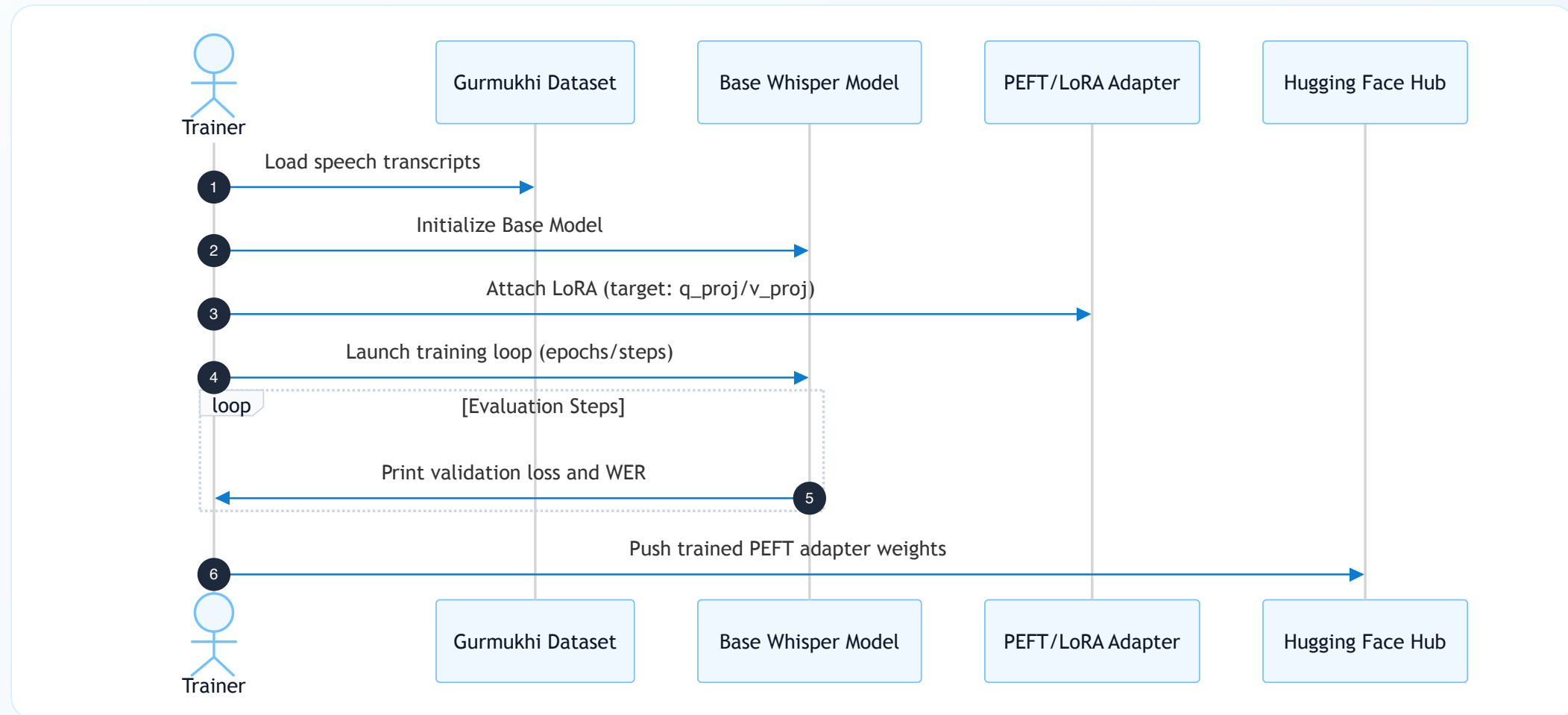
03. Fine-Tuning Setup & Training — Use Case

June 12 – June 17 (Day 5 – Day 9) | PEFT Adapter config and checkpoints export goals



03. Fine-Tuning Setup & Training — Sequence Diagram

June 12 – June 17 (Day 5 – Day 9) | Training epoch execution sequence



04. Speaker Diarization & Audio Splitting — Summary

June 18 – June 20 (Day 10 – Day 12) | 3 Days Active

We integrated Pyannote speaker tracking, combined it with noise reduction, and built diarization-based audio splitting to isolate speaker turns.

MarauliKhurad2.m4a Turn Analysis

Speaker	Duration	Turns	Share
SPEAKER_00	201.13s	55	58.7%
SPEAKER_01	141.73s	67	41.3%

Pyannote.Audio

Audio Splitting

 [Source Code Repository](#)

Check diarization notebook here: [Speaker Diarization](#)

Stitched Audio Timeline tracks

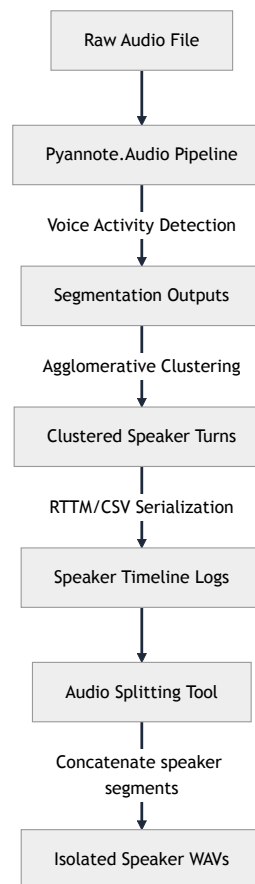


RTTM (Rich Transcription Time Marked) Export Format

```
SPEAKER MarauliKhurad2 1 0.000 0.700 SPEAKER_01
SPEAKER MarauliKhurad2 1 0.700 0.200 SPEAKER_00
SPEAKER MarauliKhurad2 1 1.400 0.600 SPEAKER_01
```

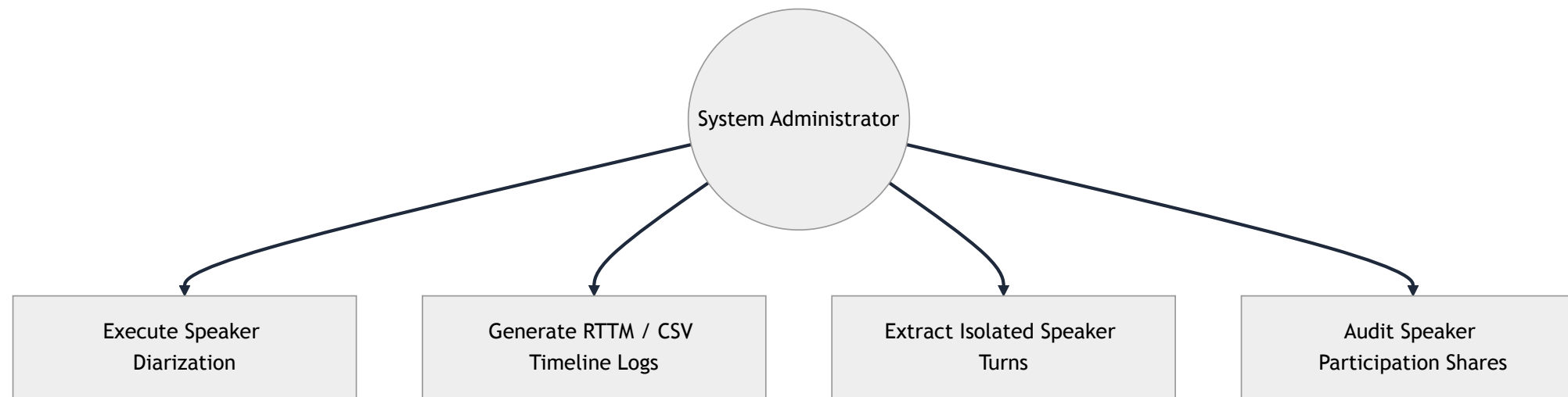
04. Speaker Diarization & Audio Splitting — Data Flow

June 18 – June 20 (Day 10 – Day 12) | Pyannote embedding pipeline & wav segment splitter



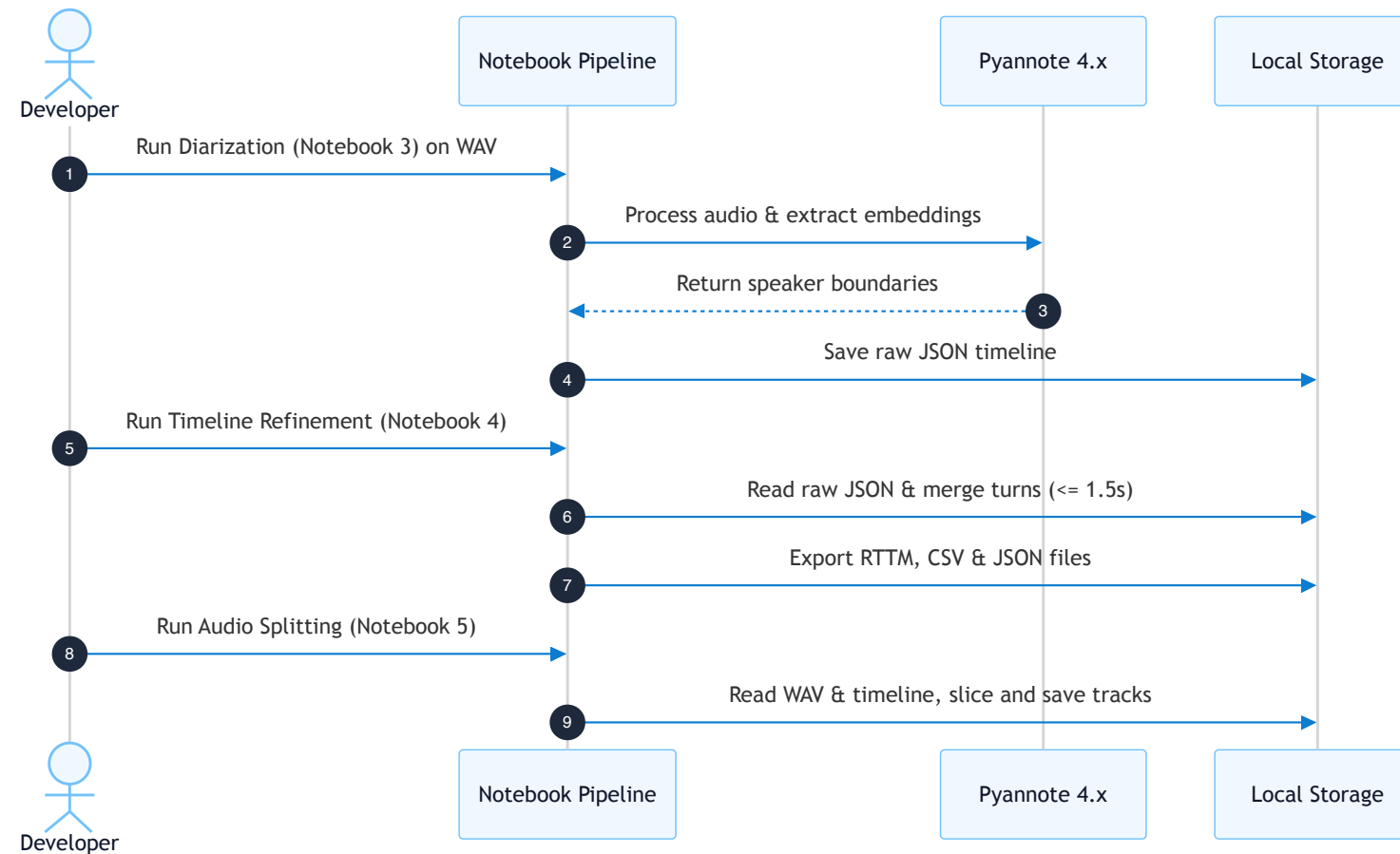
04. Speaker Diarization & Audio Splitting — Use Case

June 18 – June 20 (Day 10 – Day 12) | Turn clustering and isolation goals



04. Speaker Diarization & Audio Splitting — Sequence Diagram

June 18 – June 20 (Day 10 – Day 12) | Segment timeline exports execution flow



05. Pipeline Integration — Summary

June 22 – June 23 (Day 13 – Day 14) | 2 Days Active

We unified all processing stages (audio ingestion, noise suppression, speaker diarization, splitting, and transcription) into a unified pipeline.



UNIFIED SCRIPT PIPELINE

Unified all processing steps into a single notebook: `Final/pipeline.ipynb`. This served as the core prototype from which modular stages were later structured.

Unified Ingestion

Unified Audio Engine



Source Code Repository

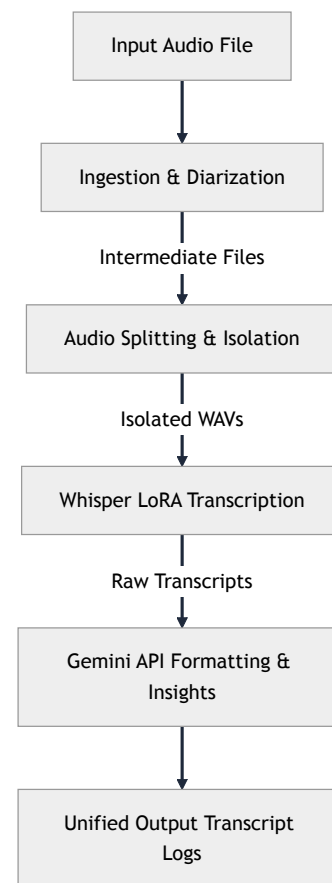
Check pipeline notebook here: [pipeline.ipynb](#)

pipeline.ipynb Sequence Flow

1. **Ingestion:** Load audio files from inputs directory.
2. **Diarization:** Predict speaker segment timestamps with Pyannote.
3. **Splitting:** Split the full audio track based on speaker segments.
4. **Transcription:** Transcribe isolated WAV segments using PEFT Whisper weights.
5. **Formatting:** Reassemble segment transcripts into speaker-labeled document.

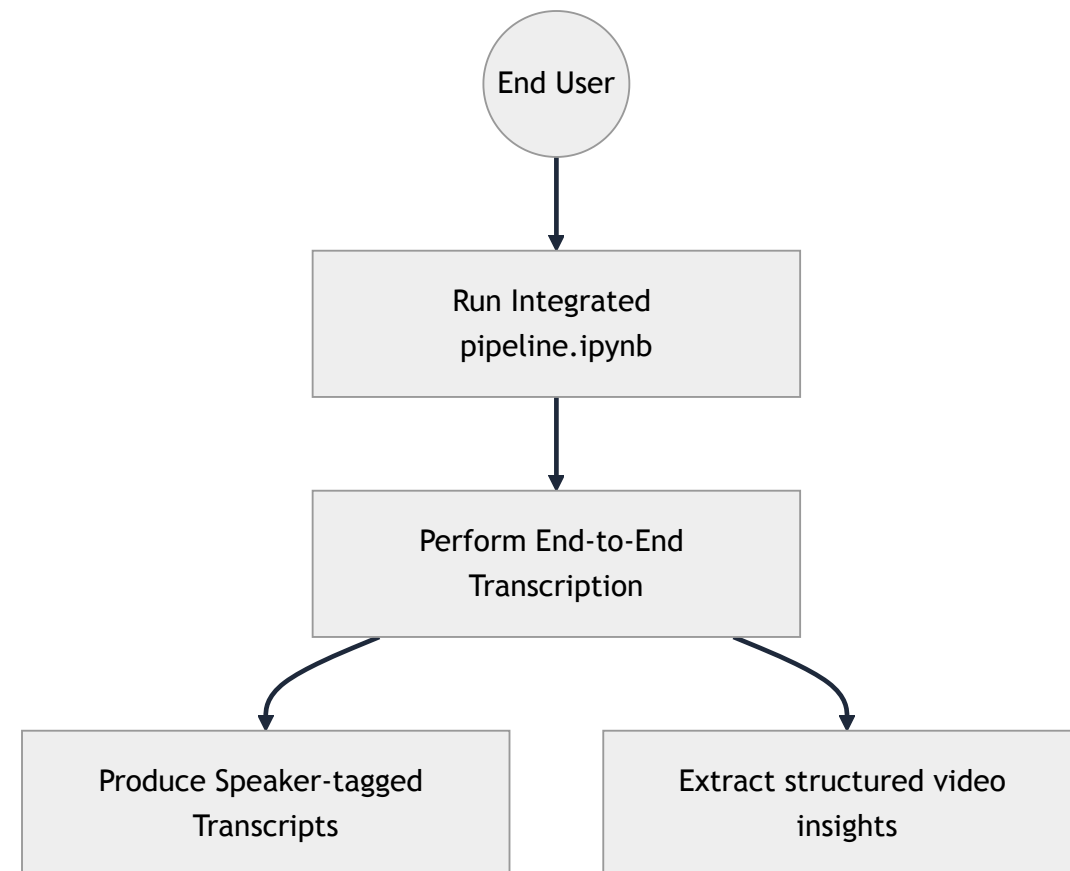
05. Pipeline Integration — Data Flow

June 22 – June 23 (Day 13 – Day 14) | Unified data pipeline execution



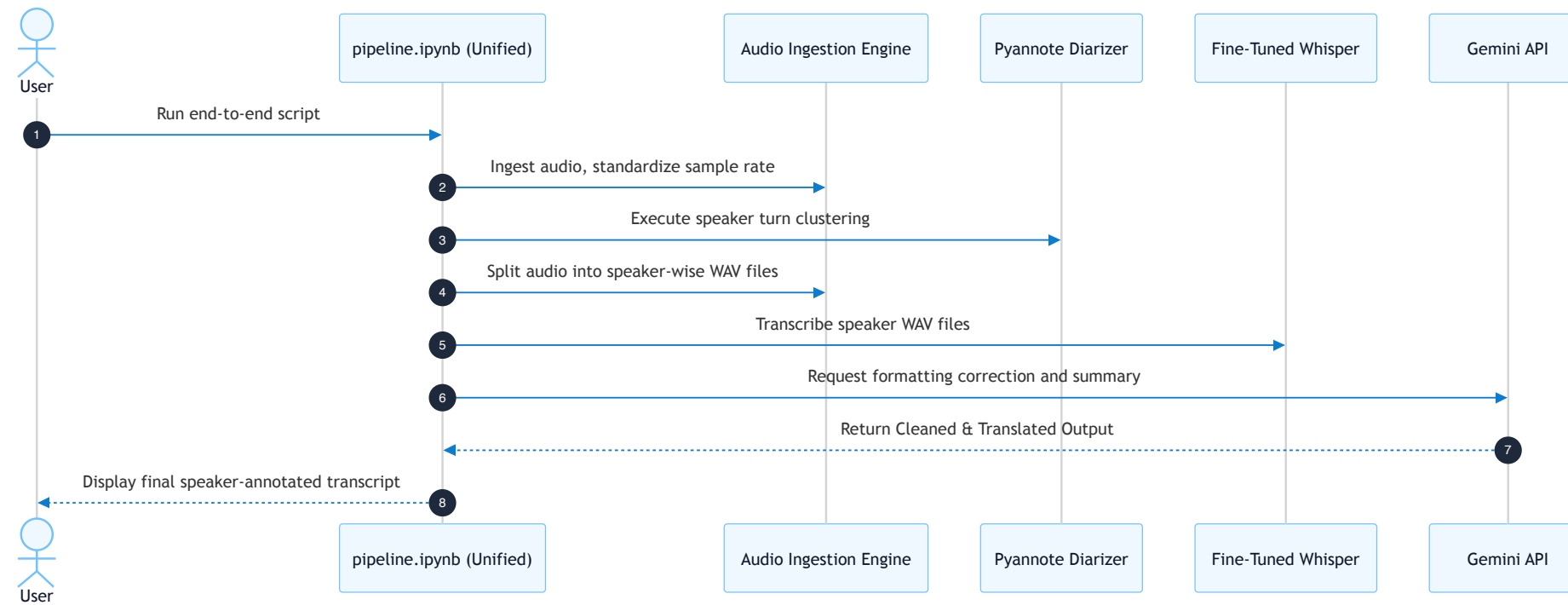
05. Pipeline Integration — Use Case

June 22 – June 23 (Day 13 – Day 14) | Unified batch transcription goals



05. Pipeline Integration — Sequence Diagram

June 22 – June 23 (Day 13 – Day 14) | End-to-end execution sequence



06. Modularization, Migration & API Fallbacks — Summary

June 24 – July 1 (Day 15 – Day 20) | 8 Days Active

 [Source Code Repository](#)

Check atomic notebooks here: [Atomic Notebooks](#)

We deconstructed the monolithic script into six atomic notebooks, migrated pipeline code to the team's official repository, and set up fallback schemas.

Decoupled Pipeline Subfolders

Intermediate files are synced across sequential steps:

01_Ingestion → 02_Noise_Suppression →
03_Diarization → 04_Timeline_Refinement →
05_Audio_Splitting →
06_Whisper_Lora_Transcription.

6 Atomic Notebooks

GDrive Intermediate Sync

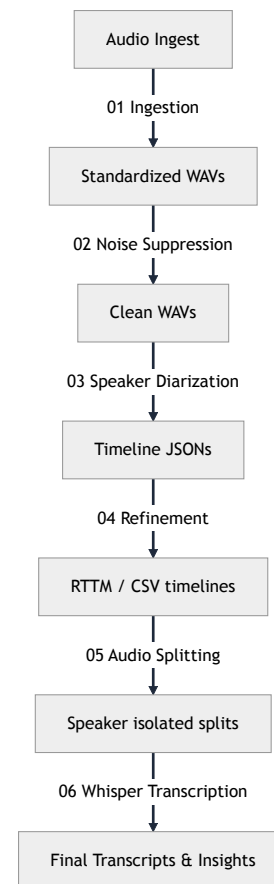
Intermediate folder structures mapped under Google Drive:

- |— 01_Ingestion/ (Standardized WAVs)
- |— 02_Noise_Suppression/ (Clean WAVs)
- |— 03_Diarization/ (Timeline JSONs)
- |— 04_Timeline_Refinement/ (RTTM & CSVs)
- |— 05_Audio_Splitting/ (Isolated Speaker Splits)
- |— 06_Whisper_Lora/ (Final Markdown Transcripts)

Collaboration migration: Successfully merged changes from fork branch into the official vicharanashala:kali_intern repository branch.

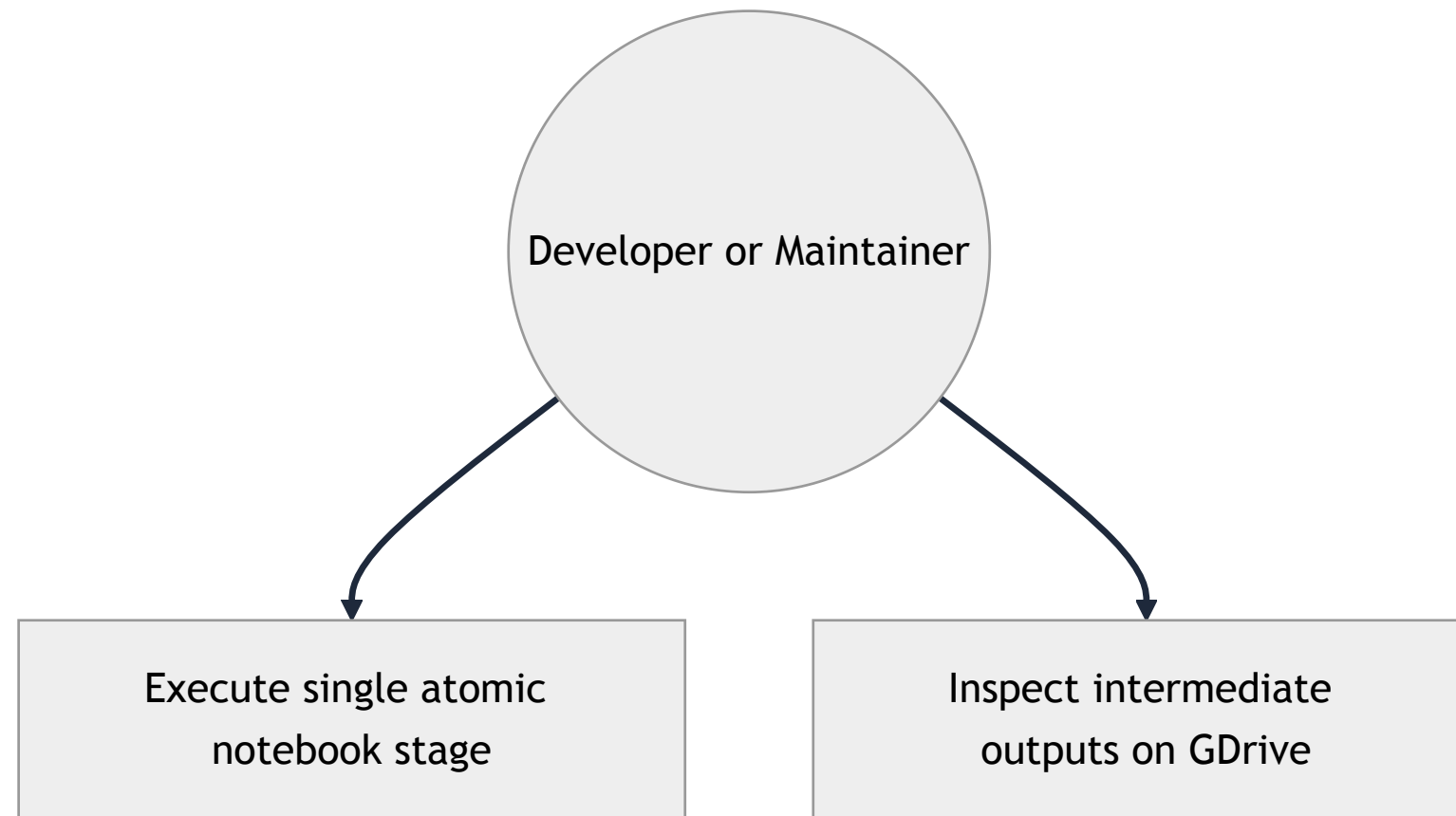
06. Modularization, Migration & API Fallbacks — Data Flow

June 24 – July 1 (Day 15 – Day 20) | Folder schemas & intermediate file exchanges



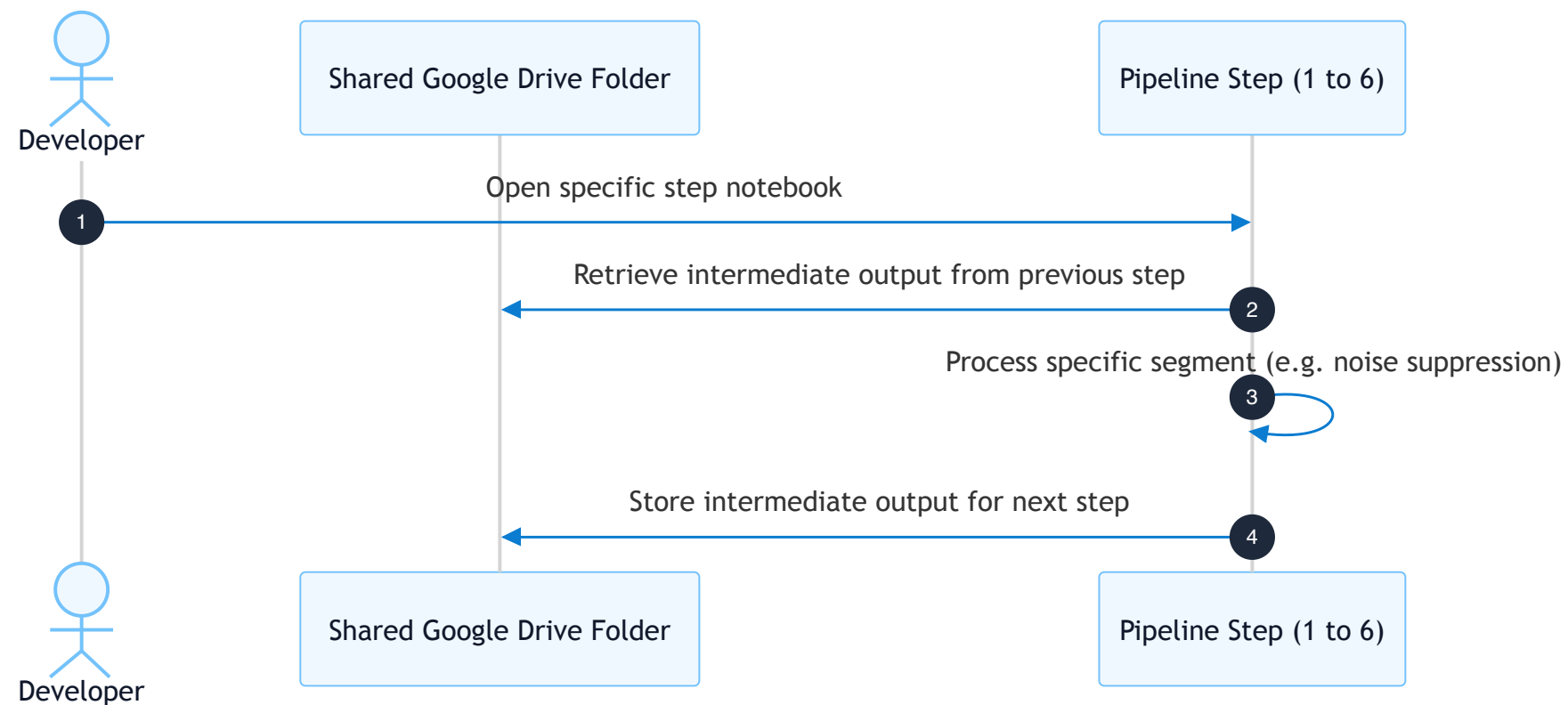
06. Modularization, Migration & API Fallbacks — Use Case

June 24 – July 1 (Day 15 – Day 20) | Step-by-step modular debugging objectives



06. Modularization, Migration & API Fallbacks — Sequence Diagram

June 24 – July 1 (Day 15 – Day 20) | GDrive synchronization & repo push sequence



07. Local LLM Scaling & Benchmarking — Summary

July 2 – July 4 (Day 21 – Day 23) | 3 Days Active

Whisper models struggled heavily with Gurmukhi/Devanagari, and even after intensive fine-tuning, the lowest WER achieved was only 0.80 (80%) due to acoustic loops. We were shocked to find that running local quantized Gemma 3n E2B dropped the final Word Error Rate to an outstanding 0.20 (20%) while running extremely fast!

 Exported Local Gemma Transcripts:

- Punjabi (Gurmukhi): [transcript_punjabi.txt](#)
- Hindi (Devanagari): [transcript_devanagari.txt](#)

Gemma 3n E2B

WER 0.20 (vs Whisper 0.80)

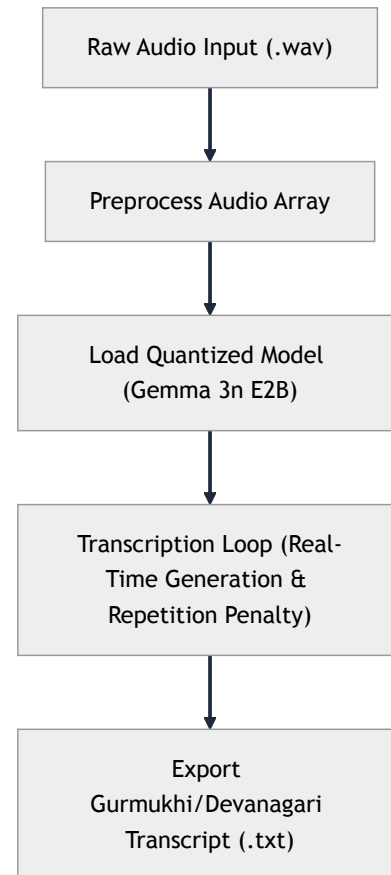
 BENCHMARKED LOCAL MODELS

Tested multiple quantized weights (Gemma 3n E2B, Gemma 3n E4B, Gemma 4 E2B/E4B) on local GPU instances.

Gemma 3n E2B was identified as the optimal sweet-spot—offering extremely low memory overhead, rapid inference speed, and superior Gurmukhi/Devanagari text token correction compared to Whisper.

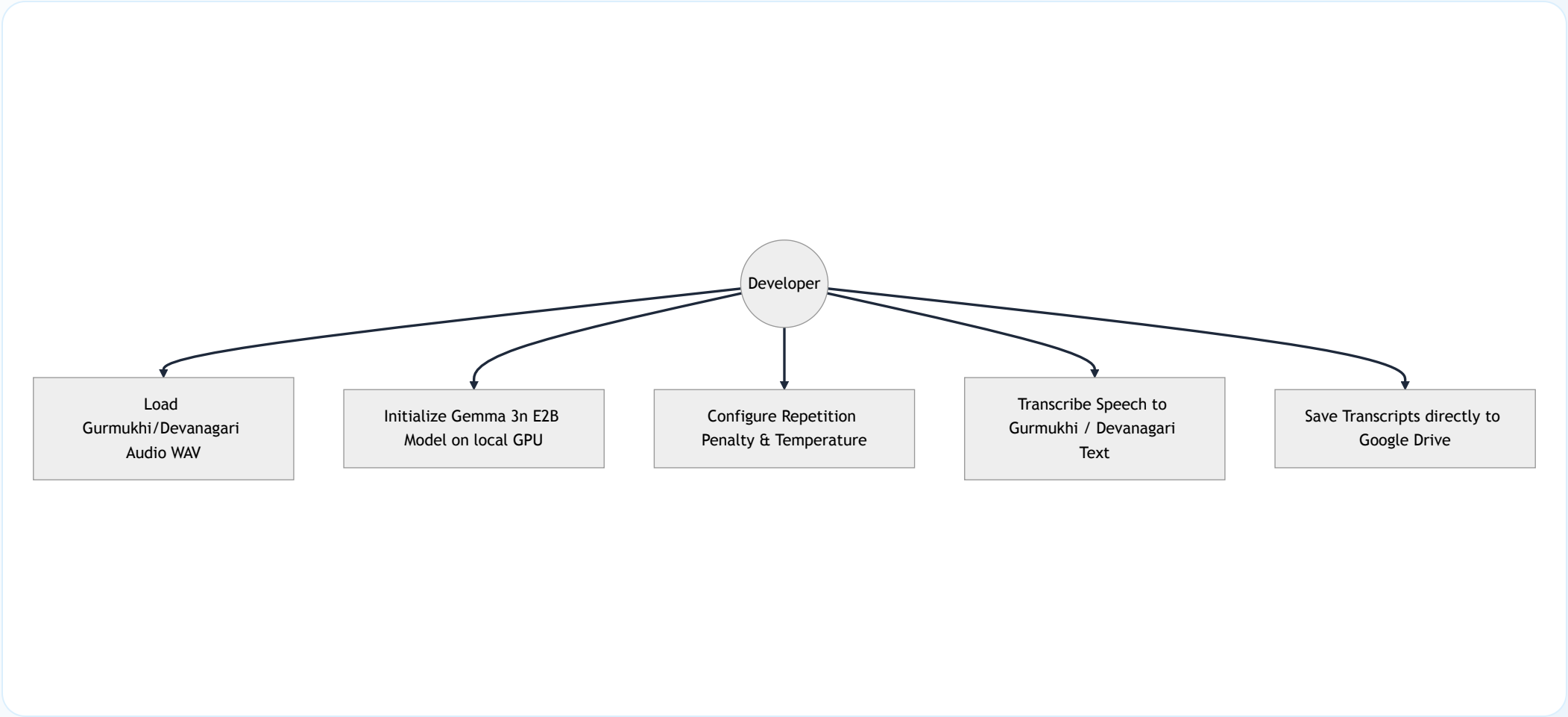
07. Local LLM Scaling & Benchmarking — Data Flow

July 2 – July 4 | Local quantized Gemma Gurmukhi/Devanagari STT notebook data paths



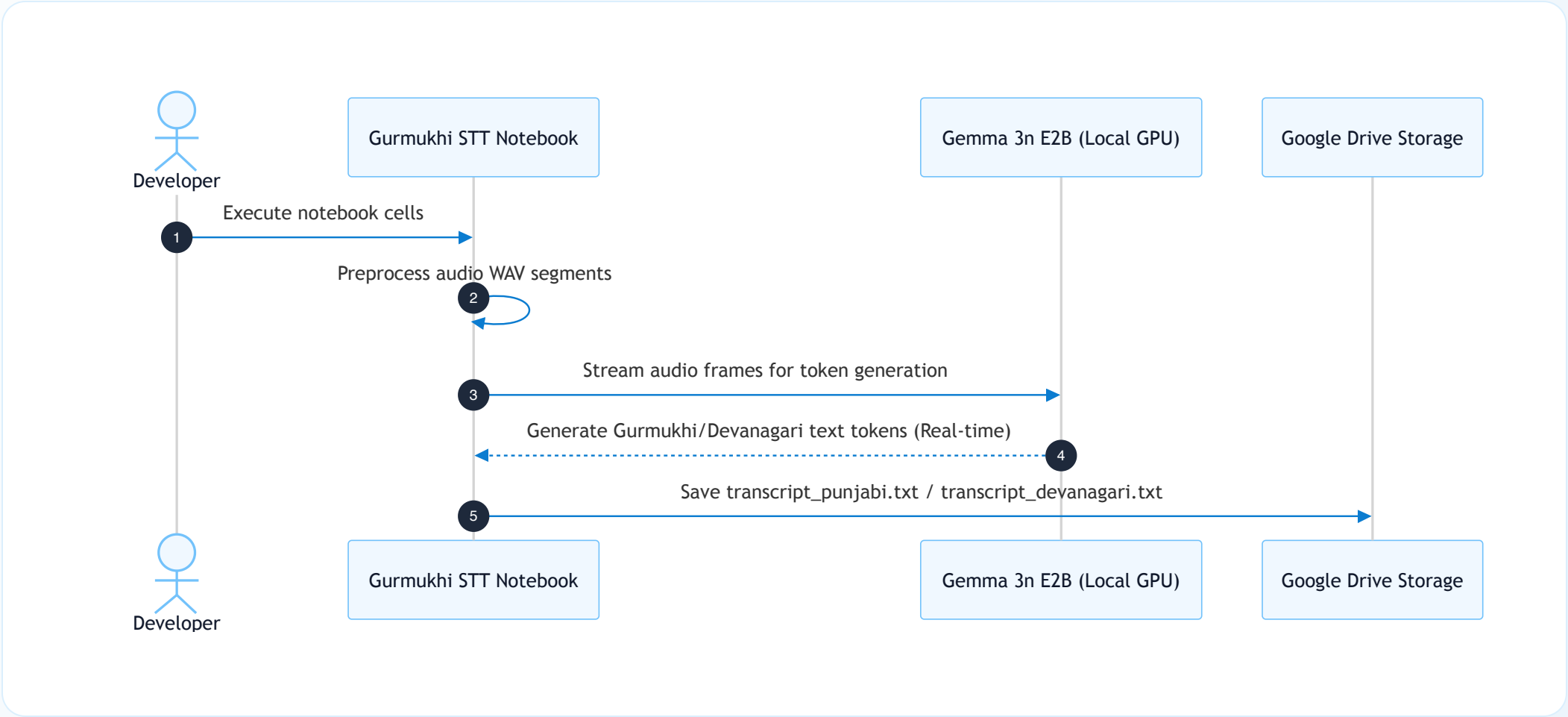
07. Local LLM Scaling & Benchmarking — Use Case

July 2 – July 4 | Local Gurmukhi/Devanagari STT notebook usage scenarios



07. Local LLM Scaling & Benchmarking — Sequence Diagram

July 2 – July 4 | Gurmukhi/Devanagari STT notebook execution sequence



08. Internship Timeline Flowchart

May 18 – July 13 | Structured vertical mapping of all 7 activities and day-to-day milestones

